SLIIT
Discover Your Future

BSc (Hons) in Information Technology Specializing Data Science

Research Project - IT4010 MLOPs Report

Group ID: R26-DS-011

Project Title: AI Based Emergency Call and SMS Triage System for Disaster Response in Sri Lankan Low Resource Local Languages

1. **IT22129758**: Multilingual Emergency Conversational AI System for Rapid Clarification
2. **IT22644480**: Multimodal Urgency Detection Using Prosodic and Conversational Cues
3. **IT22263476**: Low-Bandwidth SMS Emergency Intelligence for Disaster Response
4. **IT22504418**: Multi-Source Urgency-Based Incident Fusion, Unified Incident Records and Real-Time Operator Dashboard

		IT22129758	IT22644480	IT22263476	IT22504418
Data Pipeline	**Data Sources:**	**1. Emergency call audio corpus** 507 simulated emergency call recordings in English, Sinhala, Tamil and code-mixed Singlish / Tanglish (fire, flood, landslide, medical scenarios), each with a reference transcript. **2. Labelled NER corpus** 306 manually labelled call transcripts with 783 gold entity spans across 7 entity types: INCIDENT, ADDRESS_AREA, ADDRESS_EXACT,	**1. Simulated emergency call corpus** 267 labelled call recordings (.ogg) in Sinhala, Tamil, English, Singlish and Tanglish built around fire and flood scenarios. Class split High 130 / Medium 82 / Low 55, median duration 6.9 s. labels.csv stores language, self/others reporting, speaker gender, keywords heard and notes. Labelled by two annotators (Cohen's kappa = 0.77). **2. Real Disaster Management Centre (DMC) recordings** 59 authentic emergency calls from the November-December 2025 floods and landslides across nine districts (High 23 / Medium 14 / Low	**1. Emergency SMS dataset** 864 messages: the original 714 raw distress SMS plus 150 synthetic messages (30 per style) covering six language styles - English, Sinhala, Tamil, Singlish, Tanglish and mixed script. Target fields: incident type, location, people count and resources needed. **2. Sri Lanka gazetteer** GeoNames Sri Lanka dump (LK.txt) used to validate and resolve extracted place names. **3. Evaluation benchmarks** 500-example hallucination benchmark (250 hallucinated / 250	**1. SL-DisasterCoref benchmark** 535 multilingual call and SMS reports grouped into gold incidents (train 308 / dev 112 / test 115 reports, split by incident). Contains the report dialogues, 1,445 question-answer evidence items, 1,156 labelled evidence pairs (corroborates / contradicts / neutral / count_variance) and report pairs with hard

		PEOPLE_COUNT, TRAPPED, PLACE_TYPE and INJURY. **3. Relevance / dialogue-act corpus** 212 caller utterances labelled with 4 relevance classes (emergency-relevant, clarification response, irrelevant, non-informative) and 6 dialogue acts (provide info, confirm, deny, ask repeat, distress, other). **4. Emergency ASR dataset** emergency_asr_si_ta_en: 445 training and 49 validation audio-transcript pairs (en-US, si-LK, ta-IN) used to fine-tune Whisper for Sinhala emergency speech. **5. Live sessions** Browser-microphone and Twilio phone calls logged as per-session JSON (628 live session logs, 276 final emergency reports), used	22). Annotated before any model was applied and used only for external validation, never for training. **3. Lexicon resources** Multilingual urgency / context keyword lists, extended with TF-IDF keyword discovery on Sinhala and Tamil transcripts.	safe) plus a realistic v2 set; 26-case MPDS validation and paraphrase benchmarks; 24-case distress benchmark; field extraction accuracy benchmark. **4. Live SMS** Real messages received through a Twilio SMS number and the web chat demo.	negatives (adjacent houses, same place at a different time, cross-lingual pairs). **2. Live upstream reports** Real-time reports from Component 1 (call turns, extracted slots and Component 2's urgency verdict) and Component 3 (SMS conversation and MPDS code) through the c4.v2 contract. **3. Geographic sources** Google Places API (cached), a Sri Lanka gazetteer, area alias / district tables and an emergency contacts list. **4. Teacher labels** GPT-4o-mini field extractions (guard-accepted only) used to train the offline mT5 student.

		for error analysis and regression testing.			
	Data Preprocessing:	**1. Audio normalisation & STT clean-up** Recordings converted to 16 kHz mono WAV. Whisper decoding uses an emergency initial prompt, beam size 5 and a segment log-probability filter; an STT guard drops hallucinated segments. **2. Language labelling** Transcripts normalised to six language codes (en-US, si-LK, ta-IN, si-en, ta-en, si-ta) and used to train a code-mix language classifier. **3. NER data preparation** Gold spans aligned to tokens by character overlap (773 / 783 spans = 98.7% hit rate) and converted to BIO tags. Seeded (42) language-balanced split: 212 train / 43 dev / 51 test, with	**1. Audio cleaning** Spectral-gating noise reduction (noisereduce), resampling to 16 kHz and silence trimming (top_db = 20), saved as clean WAV. **2. Caller isolation (DMC)** pyannote.audio speaker diarization separates the operator from the caller. Caller turns are rebuilt from the diarization segments and checked manually on a verification worksheet. **3. Acoustic features (91)** 40 MFCCs (mean and std), pitch/F0 mean and std, jitter, RMS energy mean / std / slope / gradient, zero-crossing rate, spectral centroid, speaking rate and speaking-rate change. **4. Linguistic / context features (~20)** Taken from the ASR transcript (Sinhala: Component 1's multilingual faster-whisper; Tamil / English: base Whisper). Features: keyword and context scores, repetition rate, ASR-	**1. LLM cleaning chain** raw -> normalised -> language -> corrected -> enhanced -> grammar-fixed. Every LLM rewrite is wrapped in a hallucination guard (similarity threshold, word-growth limit, grounded content words only). **2. Noise taxonomy** Each message is tagged with its noise types by diffing consecutive stages. Broken grammar affects 83.7% of messages, spelling errors 37.2% and code-mixing 40.4%. **3. Adaptive router** Predicts which cleaning stages a message needs (emoji, symspellpy and a corpus-bootstrapped Singlish/Tanglish fallback): 58.3% fewer LLM calls (2,592 -> 1,080). **4. Fine-tuning data** Input-output JSONL pairs: 777 / 87 (v1) and 1,029 / 87 (v2), with 8x oversampling of messages that contain a location.	**1. Input adapter (Step 1)** Call and SMS schemas mapped to one Pydantic C4Report: turn parser, timestamp normalisation (including Excel serials), channel, incident type, MPDS / urgency to an ordinal value. Language detected by a character 1-4-gram TF-IDF + Logistic Regression model (DEV accuracy 0.973). **2. Extraction and reconciliation (Steps 1b-1e)** Upstream quality flags; question-anchored answer recovery with a 5-language number normaliser; mT5 student extraction with caller-span and

		oversampling of rare entities. **4. Relevance data** Weak labelling followed by manual review, split 142 / 30 / 40. **5. Numeral normalisation** Sinhala, Tamil and Singlish number words converted to digits; people-count spans repaired before slot filling.	quality score and a fire/smoke match. **5. Leakage control** Labels frozen (labels_frozen_20260817.csv with SHA-256) before experiments; fixed stratified 5-fold assignments; scaler and SMOTE fitted inside each training fold only.		number checks; three-source reconciliation. Upstream values are never corrected, and missing is None, never 0. **3. Evidence pool (Step 2)** One verbatim evidence item per caller turn (never translated), categorised by Model C. **4. Leakage rule** Dataset labels live only in report meta, which no pipeline stage may read. Synthetic adjacent-house reports are training data only.
	Data storage and versioning	**1. Stage-wise storage** Raw audio in audio/input. Each pipeline stage writes its own folder (output -> normalized_output -> extracted_output -> with_offtopic -> with_missing_fields ->	**1. Data layout** Raw calls in audio/; cleaned audio in audio/clean/ (auto-generated); feature matrices in data/features.csv and features_v2.csv. **2. Frozen snapshot** frozen_v1/ holds the v1 baseline artifacts (features, labels, DMC	**1. Pipeline dataset** clean_dataset_full_with_broken.json keeps, for every message, all stage outputs, noise tags, router prediction, risk scores, GPT-4o cross-check and mT5 extraction. **2. Model artifacts**	**1. Immutable dataset versions** Dataset versions v2 to v6 are each built by a script from the previous one and never edited in place

		with_questions -> with_tts), so every call can be traced through all stages. **2. Dataset versioning** Train / dev / test JSONL splits with a manifest.json recording the seed, counts and language / entity distributions. **3. Model registry** Models in models/ (xlmr_proposed_v2, xlmr_relevance, bilstm_crf_ner.pt, crf_ner.joblib, language classifier, piper_sinhala, faster-whisper-si). best_ner_meta.json and best_relevance_meta.json record the deployed model, its metrics and the deploy date. **4. Source control** Metrics stored as JSON in results/. Code tracked in Git; API keys kept in .env	metadata, diarization manifests, pipeline scripts) with MANIFEST.json storing the SHA-256 and size of each file. These are never modified; v2 uses new filenames. **3. Model artifacts** models/urgency_rf_v1.joblib, urgency_text_only_v1.joblib and urgency_acoustic_only_v1.joblib. candidate_pipeline_freeze.json/.txt record the locked 111-feature order and settings. **4. Source control** Git repository R26_DS_011. Audio, models and generated reports are git-ignored; results kept under reports/pp1, reports/pp2 and reports/pp2_finalized_reports.	Fine-tuned model in mt5-sms-emergency-final/ (safetensors, tokenizer, config), also archived as a zip. Fine-tuning JSONL files kept per version (v1, v2). **3. Results and queue** Benchmark results saved as JSON; completed cases in dispatch_queue.json, re-exported sorted by urgency as JSON and CSV. **4. Source control** Git; .env with API keys git-ignored (.env.example provided).	(v5 is the working version). **2. Artifact provenance** Every trained model is installed under learned/artifacts/ with PROVENANCE.json, SHA-256 hash and Colab run ID (e.g. mT5 run_20260929_1235, LaBSE run_20260928_1138, mDeBERTa run_20260930_1404). Superseded models are kept as backups. **3. Data packages** Colab training data exported as zip packages with a manifest of SHA-256 hashes. **4. Live stores** Live reports in data/live_reports.jsonl and incidents in <store>_incidents.json; both survive a restart. Experiment outputs in

		(git-ignored), with a .env.example template.			outputs/stepN; outputs, cache, data and artifacts are git-ignored.
Model Development	**Model Selection**	**1. Speech-to-text** faster-whisper (CTranslate2, int8). Fine-tuned Whisper-small for Sinhala (faster-whisper-si), Whisper-small for English / Tamil, Whisper-tiny for language detection. **2. Emergency NER** CRF (F1 0.275) and BiLSTM-CRF (F1 0.278) baselines vs fine-tuned XLM-RoBERTa-base (proposed, P 0.50 / R 0.38 / F1 0.429). XLM-R deployed. **3. Utterance relevance** Logistic Regression (F1 0.783) and SVM (F1 0.765) vs fine-tuned XLM-R (F1 0.819). XLM-R deployed. **4. Clarification policy**	**1. Classifier** Random Forest chosen after the PP1 model comparison; gradient boosting and stacking were tested and rejected. **2. Baselines** Early-fusion v1 (simulated 64.0% / DMC 34.2%), text-only (DMC 45.8%) and acoustic-only (DMC 33.6%). **3. Proposed - late fusion** Final probability = 0.5 x P(acoustic) + 0.5 x P(text). Locked 2026-08-30. Accuracy / macro-F1: simulated 66.4% / 0.639; DMC scope-matched 80.0% / 0.753; DMC full 44.8% / 0.431. High-class recall 75.9% / 88.6% / 57.4%. **4. Negative results** Late fusion beat 7+ other fusion strategies (early fusion, CV-tuned weights, per-class weights, confidence gating). 11+ other tuning approaches (calibration, augmentation, hyperparameter	**1. Hybrid architecture** Rule-based NLP and TF-IDF classifiers (LR / RF / SVM / NB) were replaced by an LLM pipeline (GPT-4o-mini), used only where rules failed: extraction, distress detection and a severity safety net. Memory, answer parsing, question selection and priority stay deterministic. **2. Hallucination detector** Content-word overlap (F1 0.92) and embedding similarity (F1 0.77) vs NLI entailment with multilingual-MiniLMv2 mnli-xnli (P 0.95 / F1 0.97). NLI selected. **3. Offline model** Fine-tuned mT5-small (~300M parameters, 101 languages): incident accuracy 82.76%. **4. Priority** MPDS chosen over START because MPDS is built for remote triage. Codes ECHO -> DELTA -> CHARLIE -> BRAVO -> ALPHA -> OMEGA.	**1. Rule for every step** Baselines vs proposed model, settings chosen on DEV, TEST used once. **2. Learned components** Extraction: mT5-small student, DEV 0.788 vs upstream 0.155. Semantic similarity: fine-tuned LaBSE, TEST AUC 0.946 vs 0.846 off-the-shelf vs 0.621 char TF-IDF. Evidence judge: mDeBERTa-v3-base, 4 classes, TEST macro-F1 0.718. Geography: calibrated Logistic Regression (Brier 0.094) vs GBM. Time: per-type spline Logistic Regression. **3. Fusion and decision**

		Adaptive multi-agent policy (ASK_DIRECT / ASK_SOFT / ASK_REFINE / NO_QUESTION) vs a fixed per-field template: question diversity 0.222 vs 0.040, 100% safety pass. Readiness uses a protocol- and confidence-aware assessor (EENA/NENA, OASIS CAP) instead of a null-field check. **5. Text-to-speech** Language-aware hybrid TTS (Piper Sinhala, MMS, Edge-TTS) vs English-only telephony voice: language match 100% vs 41%. GPT-4o-mini used only for optional calm phrasing, with template fallback.	search, etc.) were rejected with documented reasons.	Keyword + LLM safety net raised paraphrase accuracy from 57.7% to 73.1% and cut dangerous under-triage from 11 to 1 of 26 cases.	Gradient boosting (depth 3; DEV CV log loss 0.069 vs LR 0.079) gives a calibrated P(same incident), TEST AUC 0.983. Cost-based threshold 3:1 at incident level. **4. Final live results (TEST)** False merge 0.9%, merge precision 98.1%, link recall 70.7%, B-cubed F1 0.864.
	Model Training	**1. Framework** PyTorch with Hugging Face Transformers, Datasets and Evaluate; seqeval for entity metrics, jiwer for WER/CER.	**1. Pipeline** scikit-learn + imbalanced-learn: StandardScaler -> SMOTE (random_state 42) -> RandomForest (200 trees, min_samples_leaf 2, class_weight balanced, seed 42).	**1. Platform constraints** OpenAI, Gemini and Azure fine-tuning were all blocked (closed platform / quota / access). **2. mT5 fine-tuning**	**1. GPU training on Google Colab (T4, fp32)** mT5-small student: lr 3e-4, 30 epochs, seed 0. LaBSE: CoSENT contrastive loss with

		2. XLM-R NER xlm-roberta-base with optional CRF head; 16 epochs, batch 4, lr 2e-5, early stopping (patience 4), seed 42, rare-entity oversampling / class weights. Best checkpoint chosen by dev F1. **3. XLM-R relevance** xlm-roberta-base with relevance and dialogue-act heads; AdamW lr 2e-5, batch 8, 8 epochs, max length 160, linear warm-up schedule. **4. Whisper ASR** openai/whisper-small with Seq2SeqTrainer; lr 1e-5, batch 2 x gradient accumulation 8, 3 epochs. Converted to CTranslate2 for faster-whisper inference. **5. Baselines** sklearn-crfsuite CRF, BiLSTM-CRF (pytorch-crf),	One pipeline per branch (acoustic, text), then late fusion. **2. Validation** Stratified 5-fold cross-validation, scored by macro-F1; external test on the DMC calls. **3. Explainability** SHAP summary and per-class beeswarm plots, feature importance, per-language breakdown and confusion matrices. **4. Freeze-before-test** Feature order, model, scaler and thresholds frozen before DMC evaluation, so real-world results were never used for tuning.	google/mt5-small, full-parameter fine-tuning on Google Colab (T4 GPU) with Hugging Face Transformers. LoRA dropped after a reproducible NaN-loss bug. lr 1e-3, batch 8, 10 epochs; evaluated on 87 held-out messages. **3. Prompt and guard tuning** LLM prompts and guards iterated against the benchmarks, e.g. the distress prompt improved from F1 0.92 to 1.00 with no loss of recall.	hard negatives, lr 2e-5, 3 epochs. mDeBERTa judge: class-weighted cross-entropy, both pair orders, lr from {1e-5, 2e-5, 3e-5} and epochs 1-8 chosen on DEV. **2. Health checks** No NaN, loss decreasing, tiny-overfit test, shuffled-label control, 3 seeds agree, no split leakage. **3. Laptop verification** SHA-256 check, same data, CPU outputs equal Colab outputs, metrics reproduced -> --install. **4. CPU models (scikit-learn)** Model C, language, geography, temporal and fusion models, tuned with 5x5 CV and cross-fitted by incident. Decision rules

		TF-IDF + Logistic Regression / SVM.			declared before results.
Model Integration	**Tools used**	**1. ML / NLP / speech** Python 3, PyTorch, Transformers, Datasets, faster-whisper / CTranslate2, scikit-learn, sklearn-crfsuite, pytorch-crf, seqeval, jiwer, librosa, soundfile. **2. Speech output** piper-tts, MMS-TTS, edge-tts, gTTS. **3. Backend and telephony** Flask + Flask-CORS, Twilio Voice, OpenAI API (optional), python-dotenv. **4. Development** Git / GitHub, VS Code, Google Colab.	**1. Audio and speech** librosa, soundfile, noisereduce, openai-whisper, faster-whisper, pyannote.audio. **2. ML and analysis** pandas, NumPy, scikit-learn, imbalanced-learn, SHAP, joblib, matplotlib, seaborn. **3. Serving and development** Flask (web demo), Git / GitHub, VS Code.	**1. LLM and NLP** OpenAI API (gpt-4o-mini, gpt-4o), Hugging Face Transformers, Datasets, Accelerate, PyTorch, multilingual NLI model, deep-translator, symspellpy, emoji, langdetect, indic-transliteration. **2. Backend and messaging** Flask, Twilio SMS, GeoNames. **3. Testing and development** pytest, Git / GitHub, Google Colab, VS Code.	**1. ML / NLP** PyTorch, Transformers, sentence-transformers (LaBSE), mDeBERTa-v3, mT5, scikit-learn, indic-transliteration. **2. Data and validation** Pydantic, pandas, openpyxl. **3. Services** Flask REST API, Google Places API, OpenAI API (teacher / optional GPT step), Leaflet + OpenStreetMap for the dashboard map. **4. Development** pytest (28 test modules), Git / GitHub, Google Colab.
Model Deployment	**Testing Environments**	**1. Offline evaluation** Held-out test splits (NER 51, relevance 40) with per-language and per-entity reports.	**1. Cross-validation** 5-fold stratified CV on the simulated corpus. **2. External validation**	**1. Benchmarks** Hallucination, MPDS validation / paraphrase, distress and extraction accuracy benchmarks. **2. Unit and integration tests**	**1. Unit tests** pytest suite covering adapter, extraction, evidence, geography, temporal, fusion,

		2. Pipeline regression Full pipeline re-run over 502 calls; scripted checks for triage routes, numeral handling and Sinhala/Tamil TTS. **3. Simulated and live calls** simulate_live_call_loop.py and live microphone demo on a Windows 11 laptop (CPU, int8). **4. End-to-end** Panel console with the urgency bridge (Component 2) and publishing to Component 4. Dropped-call safety nets tested: auto-commit after 90 s, idle finalise after 120 s.	Real DMC calls, evaluated as the full set (n=59) and a fire/flood scope-matched subset (n=11). **3. Diagnostics** Ablations (acoustic groups, context features, temporal) and a domain-shift analysis (PCA of simulated vs DMC acoustics). **4. Integration** Offline bridge tests (test_bridge_offline.py, test_wiring_dryrun.py). The web demo was verified byte-identical to the live pipeline.	pytest unit tests plus full end-to-end conversation testing, which caught bugs unit tests missed. **3. Live testing** Real SMS from a phone through Twilio, and the local web demo. **4. Training environment** Google Colab (GPU) for fine-tuning; local Windows laptop for everything else.	decision and revision; checks include that urgency never enters the fusion features. **2. Offline replay** DEV / TEST reports replayed in arrival order; final evaluation averaged over 10 fold shuffles. **3. Live integration check** live_integration_check starts a real C4 server and publishes Component 1 and Component 3 sessions through the shared client. **4. Demo page** /demo builds calls or SMS turn by turn against a separate test store.
	Deployment Platform	**1. Current prototype** Flask panel console on localhost:5010 with browser mic, live slot /	**1. Web demo** Flask app on localhost:5000: upload or record a call and get High /	**1. Web demo** Flask web chat demo on localhost:5020. **2. SMS gateway**	**1. API** Flask REST API on localhost:8000: POST /

		policy view and TTS playback. **2. Telephony** Twilio Voice webhooks (/voice/incoming, /voice/turn, /voice/continue) exposed through a public tunnel URL. **3. Compute** CPU-only inference (int8 Whisper); unified launcher on localhost:5050. **4. Production target** On-premises server at an emergency call centre (e.g. DMC 117 hotline) with a GPU for speech-to-text.	Medium / Low with confidence and transcript. **2. In-process service** Runs inside Component 1's call agent through integration/urgency_bridge.py on CPU. It does no speech recognition of its own at inference.	Twilio SMS webhook (/sms) on localhost:5021 exposed through a public tunnel. **3. Compute** CPU for local models (mT5, NLI); OpenAI API in the cloud for LLM steps. **4. Unified access** Shared launcher on localhost:5050.	GET /v2/reports, GET /v2/incidents, /health. **2. Operator views** /operator (testing view), /dashboard (incident map, KPIs, timeline, fused records) and /demo. **3. Compute** CPU-only inference; Google Places and OpenAI optional and switchable.
	Deployment Method	**1. Turn pipeline** Audio -> STT -> language ID -> NER + relevance -> dispatch readiness -> clarification policy -> TTS reply. Heavy models load lazily, with warm-up at start.	**1. Per-turn classification** classify_urgency_late_fusion_async() runs on each confirmed caller turn (audio + Component 1's transcript); warmup() preloads models. **2. Call-level verdict** end_call() combines turns (mean probability + peak severity),	**1. Message flow** Each incoming SMS -> run_multi_agent -> memory agent -> follow-up question returned as TwiML. Fatigue mode switches to numbered choices. **2. Case completion**	**1. Contract** Versioned c4.v2 contract; extra fields rejected (HTTP 422) so contract drift is caught. **2. Processing** The reply returns Steps 1-3A within the

		2. Outputs Final emergency JSON saved per call and published to Component 4 through the shared c4_client (fire-and-forget, 20 s timeout). Urgency requested from Component 2 for every confirmed caller turn. **3. Configuration** Ports, model names, TTS engine and maximum turns set through .env, with no code changes.	including dropped calls through an idle-timeout. The verdict (label, confidence, probabilities) is added to the final emergency JSON and forwarded to Component 4 for display. **3. Configuration** Project location set with the URGENCY_ROOT environment variable.	MPDS priority assigned -> case added to the sorted dispatch queue (JSON / CSV) -> published to Component 4 (conversation + MPDS code). **3. Abandoned conversations** An abandonment watchdog publishes partial cases when the sender stops replying.	upstream 20 s budget. A background worker then runs Steps 3B-8, decides MERGE or SEPARATE and attaches the report to an incident (5B costs about 0.3 s per aligned pair). **3. Dashboard** Polls the API every 5 s. Environment flags (C4_LLM_ENABLED, C4_5B_BACKGROUND, C4_PLACES_LIVE) turn optional parts on or off.
Future Enhancements	**Model Improvement:**	**1. Better extraction** Larger labelled NER corpus (current F1 0.43) and active learning from auto-labelled live sessions. **2. Telephony ASR** Fine-tune Whisper on real 8 kHz telephony audio (DMC calls) and move to GPU / streaming STT	**1. ASR quality** Domain-fine-tuned ASR on real telephony audio (the main remaining gap on DMC). **2. Coverage** Extend the simulated corpus beyond fire / flood (landslide, medical) and get a native-speaker re-review of the 'unclear' DMC calls. **3. Load testing**	**1. Multi-user memory** Per-phone-number conversation memory in a database (currently one global conversation). **2. Less API dependency** A larger fine-tuned offline model to reduce reliance on the OpenAI API. **3. Conversation handling** Conversation recovery and contradiction handling (e.g. '2 people' -> 'actually 5').	**1. Fewer false splits** More cross-lingual same-incident training data to reduce false splits (28% on TEST) and better handling of the adjacent-house case. **2. Production storage** Move the JSONL stores to a database (e.g.

		(currently 50-60 s per turn on CPU). **3. Scale** Support concurrent calls. **4. MLOps** MLflow / DVC experiment tracking, Docker packaging, and latency / accuracy monitoring.	Measure concurrent multi-caller load before real deployment. **4. Models and monitoring** Try learned acoustic embeddings (e.g. wav2vec2); add drift monitoring and scheduled retraining as real calls accumulate.	**4. Validation and speed** Native-speaker review of triage keyword translations, a larger distress benchmark, and latency trimming using the adaptive router.	PostgreSQL) with concurrent workers. **3. Feedback loop** Retrain from operator-confirmed incidents, with drift monitoring and MLflow tracking. **4. Geography** Improve location resolution for Sinhala / Tamil script (Places is 53-58% within 1 km vs 83% for Latin script).